Experiment – 4

Title of the Experiment: Understanding NumPy Broadcasting and Solving Numerical Problems Using Arrays

Aim of the Experiment: To understand NumPy broadcasting rules and apply them to solve numerical problems and simple systems of equations by broadcasting scalars and vectors across arrays.

Software Required: Python 3.x, Jupyter Notebook, NumPy

Algorithm:

Test Cases:

Tracing of Code:

Expected Output:

PART A: BROADCASTING BASICS

```
import numpy as np
```

Scalar broadcasting

```
data = np.array([10, 20, 30, 40, 50])
scalar = 5
print("Original Data:", data)
print("After Adding Scalar:", data + scalar)
print("After Multiplying Scalar:", data * scalar)
```

OutPut:

```
===== PART A: BROADCASTING BASICS =====

Original Data: [10 20 30 40 50]
After Adding Scalar: [15 25 35 45 55]
After Multiplying Scalar: [ 50 100 150 200 250]
```

Vector broadcasting

```
print("\n===== VECTOR BROADCASTING =====")

# 2D array
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

row_vector = np.array([1, 2, 3])
col_vector = np.array([[10], [20], [30]])

print("Matrix:\n", matrix)
print("Row Vector Broadcast:\n", matrix + row_vector)
print("Column Vector Broadcast:\n", matrix + col_vector)
```

OutPut:

```
===== VECTOR BROADCASTING =====
Matrix:
 [[10 20 30]
 [40 50 60]
 [70 80 90]]

Row Vector Broadcast:
 [[11 22 33]
 [41 52 63]
 [71 82 93]]
```

```
Column Vector Broadcast:
 [[ 20  30  40]
 [ 60  70  80]
 [100 110 120]]
```

PART B: SOLVING NUMERICAL PROBLEMS

```python
print("\n===== PART B: SOLVING NUMERICAL PROBLEMS =====")

# Solve y = ax + b
x = np.array([0, 1, 2, 3, 4])
a = 2
b = 3

y = a * x + b
print("x values:", x)
print("y = 2x + 3:", y)
```

OutPut:

```
===== PART B: SOLVING NUMERICAL PROBLEMS =====

x values: [0 1 2 3 4]
y = 2x + 3: [ 3  5  7  9 11]
```

Solve multiple equations using vectors

```python
# Solve multiple equations using vectors
a_values = np.array([1, 2, 3])
b_values = np.array([5, 10, 15])

y_values = a_values * x[:3] + b_values
print("Solutions of multiple equations:", y_values)
```

OutPut:

```
Solutions of multiple equations: [ 5 12 21]
```

PART C: EXTENDED BROADCASTING APPLICATIONS

```
print("\n===== PART C: EXTENDED BROADCASTING APPLICATIONS =====")

# Normalization using broadcasting
scores = np.array([45, 60, 75, 90, 100])
max_score = scores.max()

normalized_scores = scores / max_score
print("Normalized Scores:", normalized_scores)
```

OutPut:

```
Normalized Scores: [0.45 0.6  0.75 0.9  1.  ]
```

```
# Error computation
actual = np.array([100, 120, 140, 160])
predicted = np.array([98, 118, 150, 155])

error = actual - predicted
print("Error Values:", error)
```

OutPut:

```
Error Values: [  2   2 -10   5]
```

```
# Distance calculation (broadcasting)
points = np.array([[1, 2], [3, 4], [5, 6]])
reference = np.array([2, 3])


distances = np.sqrt(np.sum((points - reference) ** 2, axis=1))
print("Distances from reference point:", distances)
```

OutPut:

```
Distances from reference point: [1.41421356 1.41421356 4.24264069]
```